

Tutorial para Alunos Inexperientes: Consumo de API e Tratamento de Erros com JavaScript

Este tutorial foi elaborado para introduzir alunos inexperientes ao mundo do consumo de APIs (Application Programming Interfaces) utilizando JavaScript moderno, com foco na função `fetch()` e em como lidar de forma robusta com possíveis erros.

1. O que é uma API e para que serve

API é a sigla para **Interface de Programação de Aplicações** (Application Programming Interface) ⁵. Em termos simples, uma API é um conjunto de regras e protocolos que permite que diferentes softwares se comuniquem e troquem informações entre si.

Para entender melhor, imagine que você está em um restaurante:

Componente	No Restaurante	Na Programação
Você (o Cliente)	O software que precisa de dados (seu código JavaScript).	
O Garçom (a API)	O intermediário que recebe seu pedido e o leva à cozinha.	
A Cozinha (o Servidor)	O software que possui os dados e a lógica de negócio.	
O Pedido (a Requisição)	A solicitação que seu código envia para a API (ex: "Quero os dados do clima de São Paulo").	
O Prato (a Resposta)	Os dados que a API retorna para o seu código (ex: um arquivo JSON com a temperatura e umidade).	

Para que serve?

APIs servem para expor funcionalidades e dados de forma controlada. Elas permitem que desenvolvedores usem serviços complexos (como mapas, pagamentos ou dados de redes sociais) sem precisar entender ou acessar o código-fonte desses serviços.

2. Introdução ao `fetch()`

No JavaScript moderno, a maneira mais comum e recomendada de fazer requisições de rede é usando a função `fetch()` 5 .

O `fetch()` é uma função nativa do navegador que fornece uma interface para realizar requisições HTTP (como o seu "pedido" no restaurante) e processar as respostas.

Por que `fetch()` ?

- **Promessas (Promises):** Diferente de métodos mais antigos, o `fetch()` utiliza **Promises**, o que torna o código assíncrono (que não trava a execução do programa enquanto espera a resposta) muito mais limpo e fácil de gerenciar.
- **Sintaxe Simples:** Sua sintaxe é direta e intuitiva.

A sintaxe básica para uma requisição é:

JavaScript

```
fetch(url)
  .then(response => {
    // Processa a resposta
  })
  .catch(error => {
    // Trata erros de rede
  });
```

3. Requisições GET

O método **GET** é o tipo de requisição mais comum e é usado para **solicitar dados** de um recurso específico 5 . É o equivalente a "pedir o cardápio" ou "pedir o prato" sem alterar nada na cozinha.

Ao usar `fetch()` , o método **GET** é o padrão, então você não precisa especificá-lo. Basta passar a URL do recurso que você deseja acessar.

Exemplo Básico de Requisição GET:

JavaScript

```
const url = 'https://api.exemplo.com/dados/usuarios';

fetch(url )
  .then(response => response.json()) // Converte a resposta para JSON
  .then(data => {
```

```
    console.log('Dados recebidos:', data);
  })
  .catch(error => {
    console.error('Ocorreu um erro:', error);
  });
```

4. Promises e uso de `then` / `catch`

A chave para entender o `fetch()` é o conceito de **Promise** (Promessa). Uma Promise é um objeto que representa a eventual conclusão (ou falha) de uma operação assíncrona ⁵.

Uma Promise pode estar em um de três estados:

1. **Pending (Pendente):** O estado inicial; a operação ainda não foi concluída.
2. **Fulfilled (Realizada):** A operação foi concluída com sucesso.
3. **Rejected (Rejeitada):** A operação falhou.

O `fetch()` retorna uma Promise. Para lidar com os resultados, usamos:

- `.then()` : É executado se a Promise for **realizada** (sucesso). Você pode encadear múltiplos `.then()` para processar os dados em etapas. O primeiro `.then()` geralmente converte a resposta HTTP para o formato desejado (como JSON).
- `.catch()` : É executado se a Promise for **rejeitada** (falha). No caso do `fetch()`, o `.catch()` só é acionado por **erros de rede** (ex: sem conexão, falha de DNS).

Fluxo de uma Requisição:

1. `fetch(url)` : Inicia a requisição e retorna uma Promise.
2. `.then(response => response.json())` : O primeiro `.then` recebe o objeto `Response` e o transforma em uma nova Promise que resolve com os dados em formato JSON.
3. `.then(data => ...)` : O segundo `.then` recebe os dados JSON prontos para uso.
4. `.catch(error => ...)` : Captura qualquer erro de rede que tenha ocorrido durante o processo.

5. Exibição de dados dinâmicos no HTML

Após receber e processar os dados (o objeto `data` no último `.then()`), o próximo passo é exibi-los na sua página web. Isso é feito manipulando o **DOM** (Document Object Model) do HTML.

Passos Básicos:

1. **Selecione o Elemento:** Use métodos como `document.getElementById()`, `document.querySelector()`, ou `document.querySelectorAll()` para encontrar o local no HTML

onde os dados serão exibidos.

2. **Crie o Conteúdo:** Use os dados recebidos para gerar o HTML que você deseja inserir (ex: uma lista, um parágrafo, uma tabela).
3. **Insira no DOM:** Use propriedades como `element.innerHTML` ou métodos como `element.appendChild()` para injetar o novo conteúdo na página.

Exemplo de Inserção:

Se você tem um `div` com `id="resultado"` :

HTML

```
<div id="resultado">Carregando dados...</div>
```

E os dados recebidos são `data = { nome: "Usuário A", idade: 30 }` :

JavaScript

```
// Dentro do último .then(data => ...)
const resultadoDiv = document.getElementById('resultado');
resultadoDiv.innerHTML = `
  <h2>${data.nome}</h2>
  <p>Idade: ${data.idade}</p>
`;
```

6. Tratamento de erros de rede e resposta

O tratamento de erros é crucial para criar aplicações robustas. Com `fetch()`, é vital entender a diferença entre:

A. Erro de Rede (Capturado pelo `.catch()`)

Ocorre quando a requisição não consegue ser enviada ou o servidor não responde (ex: sem internet, URL malformada, falha de DNS). **Neste caso, a Promise é rejeitada e o `.catch()` é acionado.**

B. Erro de Resposta HTTP (Precisa ser Checado Manualmente)

Ocorre quando a requisição é enviada e o servidor responde, mas com um código de status que indica falha (ex: `404 Not Found`, `500 Internal Server Error`, `401 Unauthorized`). **Neste caso, a Promise é *realizada* (sucesso), e o `.catch()` NÃO é acionado.**

Para tratar erros HTTP, você deve verificar a propriedade `response.ok` ou `response.status`

dentro do primeiro `.then()` :

- `response.ok` : É `true` para códigos de status na faixa 200-299 (sucesso).
- `response.status` : Contém o código de status HTTP (ex: 404, 500).

Tratamento de Erros Completo:

JavaScript

```
fetch(url)
  .then(response => {
    // 1. Checagem de Erro HTTP
    if (!response.ok) {
      // Lança um erro para ser capturado pelo .catch()
      throw new Error(`Erro HTTP! Status: ${response.status}`);
    }
    // 2. Se tudo ok, converte para JSON
    return response.json();
  })
  .then(data => {
    console.log('Dados de sucesso:', data);
    // ... Exibir dados no HTML
  })
  .catch(error => {
    // 3. Captura Erros de Rede OU Erros lançados no .then()
    console.error('Falha na operação:', error.message);
    // ... Exibir mensagem de erro para o usuário no HTML
  });
```

Referências

[1] Tecnoblog. O que é API? Saiba para que serve e conheça exemplos práticos de aplicações. Disponível em: [URLDAFONTE_1]

[2] MDN Web Docs. Using the Fetch API. Disponível em: [URLDAFONTE_2]

[3] Mozilla Developer Network. HTTP request methods. Disponível em: [URLDAFONTE_3]

[4] MDN Web Docs. Promise. Disponível em: [URLDAFONTE_4]

Exercícios de Fixação

Para praticar os conceitos de Requisições GET, Promises, Exibição de Dados e Tratamento de Erros, você pode executar os exercícios no arquivo `exercicios.html` que acompanha este tutorial.

Exercício 3: Requisições GET

Objetivo: Entender o objeto `Response` retornado pelo `fetch()` .

Instrução: No arquivo `exercicios.html` , implemente a função `exercicio3()` para fazer uma requisição GET para a URL `https://jsonplaceholder.typicode.com/posts/1` e exibir o objeto `Response` completo no console do navegador.

Exercício 4: Promises e uso de `then` / `catch`

Objetivo: Praticar o encadeamento de `.then()` e o uso do `.catch()` para erros de rede.

Instrução:

1. **Sucesso:** Implemente a função `exercicio4_sucesso()` para buscar os dados do usuário 1 (`/users/1`), usando o primeiro `.then()` para chamar `response.json()` e o segundo `.then()` para exibir os dados JSON no console.
2. **Erro de Rede:** Observe a função `exercicio4_erro()` , que tenta acessar uma URL inválida. Verifique como o `.catch()` é acionado para lidar com a falha de conexão.

Exercício 5: Exibição de Dados Dinâmicos no HTML

Objetivo: Usar os dados recebidos da API para manipular o DOM e exibir informações na página.

Instrução: Implemente a função `exercicio5()` para buscar o post 2 (`/posts/2`). No `.then()` final, use o `document.getElementById('res-ex5')` para inserir o `title` como um `<h3>` e o `body` como um `<p>` dentro da div de resultado.

Exercício 6: Tratamento de Erros de Rede e Resposta

Objetivo: Implementar a lógica de tratamento de erros HTTP (como 404) que o `fetch()` não trata automaticamente.

Instrução: Implemente a função `exercicio6()` para buscar um post que não existe (`/posts/9999`). Dentro do primeiro `.then(response => ...)` :

1. Verifique se `response.ok` é falso.
2. Se for falso, use `throw new Error(\`Erro HTTP! Status: ${response.status}\`)` para forçar a Promise a ser rejeitada.
3. O `.catch()` deve capturar este erro e exibir uma mensagem amigável na div de resultado.

Mini-Projeto Prático: Gerador de Citações Aleatórias

Para consolidar seu aprendizado, propomos um mini-projeto prático que utiliza uma API pública e simples: a **DummyJSON Quotes API** 5 .

Objetivo do Projeto

Criar uma página HTML com um botão que, ao ser clicado, busca uma citação aleatória da API e a exibe na tela, juntamente com o nome do autor.

Detalhes da API

- **Endpoint:** `https://dummyjson.com/quotes/random`
- **Método:** `GET`
- **Resposta (Exemplo JSON):**

JSON

```
{
  "id": 10,
  "quote": "A vida é o que acontece enquanto você está ocupado fazendo outros planos.",
  "author": "John Lennon"
}
```

Estrutura do Código

Você precisará de um arquivo `index.html` e um bloco `<script>` para o JavaScript.

1. HTML (`index.html`)

HTML

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Gerador de Citações</title>
  <style>
    body { font-family: Arial, sans-serif; text-align: center; margin-top: 50px; }
    #citacao-container {
      border: 1px solid #ccc;
      padding: 20px;
      margin: 20px auto;
      max-width: 600px;
      min-height: 100px;
    }
    .citacao { font-style: italic; font-size: 1.2em; }
    .autor { margin-top: 10px; font-weight: bold; }
```

```

    </style>
</head>
<body>
  <h1>Citação do Dia</h1>
  <button id="btn-gerar">Gerar Nova Citação</button>

  <div id="citacao-container">
    <p class="citacao" id="texto-citacao">Clique no botão para carregar
uma citação.</p>
    <p class="autor" id="autor-citacao"></p>
  </div>

  <script>
    // Seu código JavaScript virá aqui
  </script>
</body>
</html>

```

2. JavaScript (Dentro da tag `<script>`)

O código deve implementar a função `carregarCitacao()` e associá-la ao botão.

JavaScript

```

const btnGerar = document.getElementById('btn-gerar');
const textoCitacao = document.getElementById('texto-citacao');
const autorCitacao = document.getElementById('autor-citacao');
const API_URL = 'https://dummyjson.com/quotes/random';

function carregarCitacao( ) {
  // 1. Limpa o conteúdo e mostra status de carregamento
  textoCitacao.textContent = 'Carregando...';
  autorCitacao.textContent = '';
  btnGerar.disabled = true; // Desabilita o botão para evitar cliques
múltiplos

  fetch(API_URL)
    .then(response => {
      // Tratamento de Erro HTTP (Status 4xx, 5xx)
      if (!response.ok) {
        throw new Error(`Erro HTTP: ${response.status}`);
      }
      return response.json();
    })
    .then(data => {
      // 2. Exibe os dados no HTML
      textoCitacao.textContent = data.quote;

```



```
        autorCitacao.textContent = `- ${data.author}`;
    })
    .catch(error => {
        // 3. Trata Erros de Rede ou Erros Lançados
        console.error('Falha ao buscar citação:', error);
        textoCitacao.textContent = 'Erro ao carregar a citação. Tente novamente.';
        autorCitacao.textContent = '';
    })
    .finally(() => {
        // 4. Garante que o botão seja reativado, independente do resultado
        btnGerar.disabled = false;
    });
}

// Associa a função ao evento de clique do botão
btnGerar.addEventListener('click', carregarCitacao);

// Carrega uma citação automaticamente ao abrir a página
carregarCitacao();
```

Este mini-projeto demonstra o ciclo completo: requisição, tratamento de sucesso, tratamento de erro HTTP, tratamento de erro de rede e manipulação do DOM.

Demonstração Prática: API em Python com Flask

Agora que você aprendeu os conceitos fundamentais de consumo de APIs, vamos demonstrar como criar uma **API simples em Python usando Flask** e como consumi-la com JavaScript.

Por que Flask?

Flask é um framework web minimalista em Python que permite criar APIs REST de forma rápida e intuitiva. É perfeito para fins educacionais e prototipagem.

Estrutura da Demonstração

A demonstração inclui dois arquivos principais:

1. **api_flask_demo.py** : Uma API simples que gerencia uma biblioteca de livros.
2. **consumir_api_flask.html** : Uma página HTML/JavaScript que consome a API.

Instalação de Dependências

Antes de executar a API, você precisa instalar Flask e Flask-CORS:

Bash

```
pip install flask flask-cors
```

Executando a API Flask

Para iniciar a API, abra um terminal e execute:

Bash

```
python3 api_flask_demo.py
```

Você verá uma mensagem indicando que a API está rodando em `http://localhost:5000`.

Endpoints da API

A API Flask oferece os seguintes endpoints:

Método	Endpoint	Descrição
GET	<code>/api/status</code>	Retorna o status da API
GET	<code>/api/livros</code>	Lista todos os livros
GET	<code>/api/livros/<id></code>	Obtém um livro pelo ID
GET	<code>/api/livros/autor/<nome></code>	Busca livros por autor
POST	<code>/api/livros</code>	Cria um novo livro

Exemplo de Resposta

Requisição:

Plain Text

```
GET http://localhost:5000/api/livros/1
```

Resposta (JSON):

JSON

```
{
  "id": 1,
  "titulo": "O Pequeno Príncipe",
  "autor": "Antoine de Saint-Exupéry",
  "ano": 1943,
  "descricao": "Uma história poética sobre um jovem príncipe que viaja por planetas."
}
```

Consumindo a API com JavaScript

Abra o arquivo `consumir_api_flask.html` em seu navegador. A página fornece uma interface interativa com abas para testar cada endpoint:

1. **Status da API:** Verifica se a API está funcionando.
2. **Listar Livros:** Recupera todos os livros.
3. **Buscar Livro:** Busca um livro específico pelo ID.
4. **Criar Livro:** Adiciona um novo livro via POST.
5. **Buscar por Autor:** Encontra livros de um autor específico.

Código JavaScript Importante

Dentro do arquivo `consumir_api_flask.html`, a função principal que faz requisições é:

JavaScript

```
function fazerRequisicao(url, metodo = 'GET', dados = null,
resultadoElementId = null) {
  const elementoResultado = resultadoElementId ?
document.getElementById(resultadoElementId) : null;

  if (elementoResultado) {
    elementoResultado.innerHTML = '<div class="status carregando">⌚
Carregando...</div>';
    elementoResultado.style.display = 'block';
  }

  const opcoes = {
    method: metodo,
    headers: {
      'Content-Type': 'application/json'
    }
  };
};
```

```

if (dados) {
    opcoes.body = JSON.stringify(dados);
}

fetch(url, opcoes)
    .then(response => {
        // Tratamento de erro HTTP
        if (!response.ok) {
            throw new Error(`Erro HTTP! Status: ${response.status}`);
        }
        return response.json();
    })
    .then(data => {
        if (elementoResultado) {
            exibirResultado(data, elementoResultado, true);
        }
    })
    .catch(error => {
        if (elementoResultado) {
            exibirErro(error, elementoResultado);
        }
    });
}

```

Esta função demonstra todos os conceitos aprendidos:

- **Requisição com `fetch()`** : Faz a chamada HTTP.
- **Tratamento de Erro HTTP**: Verifica `response.ok` e lança um erro se necessário.
- **Promises com `.then()` e `.catch()`** : Encadeia operações assíncronas.
- **Manipulação do DOM**: Exibe os resultados na página.

Fluxo Completo de Uma Requisição

1. O usuário clica em um botão na página HTML.
2. A função JavaScript correspondente é chamada.
3. `fetch()` envia a requisição para a API Flask.
4. A API processa a requisição e retorna um JSON.
5. O `.then()` recebe a resposta e verifica o status HTTP.
6. Se tudo estiver bem, o JSON é exibido na página.
7. Se houver erro, o `.catch()` captura e exibe uma mensagem de erro.

Próximos Passos

Agora que você entende como criar e consumir APIs:

1. **Experimente modificar a API:** Adicione novos endpoints ou campos aos livros.
 2. **Estenda a página HTML:** Crie novas funcionalidades, como editar ou deletar livros.
 3. **Explore APIs Públicas:** Use a API de citações do mini-projeto ou outras APIs públicas.
 4. **Aprenda sobre Autenticação:** Pesquise como proteger APIs com tokens JWT.
-

Conclusão

Parabéns! Você completou este tutorial sobre consumo de APIs e tratamento de erros. Você aprendeu:

✓ O que é uma API e para que serve ✓ Como usar a função `fetch()` do JavaScript ✓ Como fazer requisições GET ✓ Como trabalhar com Promises e encadeamento de `.then()` e `.catch()` ✓ Como exibir dados dinâmicos no HTML ✓ Como tratar erros de rede e resposta HTTP ✓ Como criar uma API simples em Python com Flask ✓ Como consumir uma API com JavaScript

Os conhecimentos adquiridos aqui são fundamentais para qualquer desenvolvedor web moderno. Continue praticando, explorando novas APIs e construindo projetos interessantes!

Referências

[5] Tecnoblog. O que é API? Saiba para que serve e conheça exemplos práticos de aplicações. Disponível em:

[5] MDN Web Docs. Using the Fetch API. Disponível em:

[5] Mozilla Developer Network. HTTP request methods. Disponível em:

[5] MDN Web Docs. Promise. Disponível em:

[5] DummyJSON. Quotes API. Disponível em:

Autor: Manus AI **Data:** Novembro de 2025 **Versão:** 1.0